
Local Prediction Segment Learning: Approximate Credit Assignment via Segmented Auxiliary Classifiers

Matthew Perlman Daniel Bendersky Sasha Sirotkin
Manning College of Information and Computer Sciences
University of Massachusetts Amherst
Amherst, MA 01003
{mperlman, dbendersky, asirotkin}@umass.edu

Abstract

We introduce **Local Prediction Segment Learning (LPSL)**, a training algorithm that achieves approximate local credit assignment by partitioning a neural network into sequential *prediction segments*. Each segment is trained using an isolated loss signal from its *prediction head*, an auxiliary linear classifier attached to the segment’s final layer. The prediction head maps the flattened output activations of the segment to a probability distribution over classes via cross-entropy loss, and is updated jointly with the segment’s parameters. Since gradients are blocked at segment boundaries, no error signal propagates between segments. The final network output is taken from the last segment’s prediction head. LPSL generalizes both backpropagation (the single-segment case) and per-layer local learning (the per-layer case), natively supports multiclass output, and integrates into arbitrary differential architectures like CNNs and transformers seamlessly. We evaluate LPSL on three controlled tasks: synthetic cluster classification with a multilayer perceptron, image classification on MNIST with a convolutional network, and modular addition with a small transformer. On all three tasks, LPSL trains models that achieve accuracy comparable to backpropagation, but often exhibit increased convergence variance. These results provide a proof of concept that segment-local credit assignment is a viable training strategy across several common architectures.

1 Introduction

Backpropagation [Rumelhart et al., 1986] is the dominant algorithm for training deep neural networks and underlies virtually all of modern deep learning. Despite its empirical success, backpropagation carries well-known theoretical tensions with established research on learning in biological neural systems. Three objections have received sustained attention. First, the **weight transport problem**: the backward pass requires a transposed copy of the forward weights to propagate error signals [Crick, 1989, Lillicrap et al., 2016]. No established biological equivalent exists for this symmetric feedback structure. Second, the **update locking problem**: all layers must wait for the complete forward and backward pass before any weights are updated, preventing the kind of continuous, parallel synaptic modification observed in biological circuits [Jaderberg et al., 2017]. Third, the **non-local credit assignment problem**: the gradient of any weight depends on the activities and weights of every downstream layer, entangling learning signals across the full depth of the network in a manner difficult to reconcile with the local nature of synaptic plasticity [Bengio et al., 2015].

These concerns have motivated a substantial body of work seeking training algorithms that operate solely on locally available information. Local layer-wise learning [Bengio et al., 2007, Nøkland & Eidnes, 2019, Belilovsky et al., 2019] trains each layer independently using auxiliary classification heads, eliminating backpropagation across layers entirely. Predictive coding networks [Rao & Ballard, 1999, Whittington & Bogacz, 2017] frame learning as inference in a hierarchical generative model,

where weight updates depend only on local prediction errors. Most recently, Hinton’s Forward-Forward (FF) algorithm [Hinton, 2022] replaces the forward-backward pass of backpropagation with two forward passes — one on positive data and one on negative data — where each layer independently optimizes a scalar *goodness* function measuring the sum of squared layer activities.

The FF algorithm’s explicit rejection of the backward pass generated considerable interest, and subsequent work has extended it to convolutional architectures [Scodellaro et al., 2023] and explored variants of the positive/negative data construction [Ortiz Torres et al., 2025]. However, the binary positive/negative framing introduces a structural limitation for multiclass problems: because goodness is a single scalar value per layer, the network must be run once per class at inference time. Extensions of FF to convolutional networks require non-trivial label embedding strategies, such as the Fourier-pattern-based spatial encoding introduced by Scodellaro et al. [2023], to inject class information across all spatial positions of a convolutional layer.

LPSL draws direct inspiration from the FF algorithm and from local layer-wise learning, while addressing several of their limitations. Rather than a scalar goodness per layer, LPSL attaches to each segment a prediction head that outputs a full probability distribution over classes. Each logit produced by the head is analogous to a class-specific goodness score, and the cross-entropy loss enforces competing goodness between classes. This eliminates the need for multiple forward passes at inference time and provides a natural multiclass objective without positive/negative data construction or spatially embedded labels.

Concretely, LPSL partitions the layers of a network into K contiguous *segments*. Each segment is assigned an independent optimizer and a lightweight linear prediction head that is trained jointly with the segment’s parameters under an isolated cross-entropy loss. A stop-gradient operation at each segment boundary ensures that no error signal propagates between segments; each segment’s backward pass is fully contained within that segment. The full forward pass remains end-to-end, so activations flow normally through the entire network, but each segment’s learning is driven exclusively by its local prediction signal.

This design achieves several desirable properties simultaneously:

- **Approximate local credit assignment.** Gradients are contained within segments; no signal traverses segment boundaries.
- **Multiclass output.** A single forward pass suffices for inference at every segment; no per-class passes or post-hoc classifiers are needed.
- **Architecture generality.** The prediction head operates on the flattened output of any segment, making LPSL applicable to MLPs, CNNs, and transformers, as well as networks containing non-differentiable components.
- **Theoretical unification.** LPSL is a strict superset of both backpropagation ($K = 1$) and per-layer local learning ($K = L$).

We evaluate LPSL on three tasks spanning three architectures. On synthetic cluster classification with an MLP, LPSL matches backpropagation accuracy while exhibiting less overfitting at long training horizons. On MNIST with a CNN, LPSL converges to competitive accuracy but with higher run-to-run variance and slower convergence. On modular addition with a small transformer, LPSL matches backpropagation’s final accuracy while converging considerably faster on average. The current scope of LPSL is particularly limited: all experiments are controlled toy tasks, run-to-run variance under LPSL is higher than under backpropagation, and sensitivity to macro-hyperparameters is greater. We argue nonetheless that successfully training non-trivial models across three distinct architectures under a locally constrained credit assignment regime is a meaningful result.

2 Related Work

2.1 Backpropagation and Its Biological Implausibilities

Backpropagation [Rumelhart et al., 1986] computes exact gradients of a global loss by propagating error signals from the output layer backwards through the full depth of the network via the chain rule. While empirically powerful, it has been criticized on several grounds relevant to this work. The weight transport problem [Crick, 1989] arises because backpropagation requires feedback weights

that are precisely transposed copies of the forward weights — a symmetric structure not observed in biological neural circuits. The update locking problem [Jaderberg et al., 2017, Nøkland & Eidnes, 2019] refers to the requirement that no layer may update its weights until the full forward and backward passes have completed. The global credit assignment problem [Bengio et al., 2015] stems from the dependency of all layer gradients on downstream computations and global loss, an opposing architecture to any biologically plausible local plasticity rules. Collectively, these issues highlight a gap between artificial and biological learning.

2.2 Local Layer-Wise Learning

A substantial line of work has explored training networks layer by layer using locally defined objectives. Bengio et al. [2007] introduced greedy layer-wise pre-training, training each layer sequentially with an unsupervised auxiliary objective as initialization for subsequent fine-tuning. Supervised variants, in which each layer receives its own classification loss from an auxiliary head, were developed in subsequent work. Belilovsky et al. [2019] demonstrated that greedy layerwise learning with auxiliary classifiers can scale to ImageNet, establishing layer-local training as viable at non-trivial scale. Belilovsky et al. [2020] extended this to convolutional networks with a decoupled training scheme. Nøkland & Eidnes [2019] showed that combining local cross-entropy loss with a local similarity matching loss can match global backpropagation on CIFAR-10 and ImageNet, with the key advantages of eliminating the backward-locking problem and reducing peak memory.

LPSL is closely related to this family of methods. The central difference is granularity: where local layer learning assigns an independent loss to each individual layer, LPSL assigns a loss to each *segment* — a contiguous block of multiple layers. This allows the algorithm to smoothly interpolate between fully local training (single-layer segments) and fully global training (one segment spanning the full network). Intra-segment gradients flow normally, capturing more representational depth per local update than single-layer methods, which have been shown to suffer from a “short-sightedness” problem as depth increases [Wang et al., 2021].

2.3 The Forward-Forward Algorithm

Hinton [2022] proposed the Forward-Forward algorithm as a replacement for the forward-backward pass. FF trains each layer with a contrastive objective: weights are adjusted to produce high summed squared activity (goodness) for positive data and low goodness for negative data. Each layer is trained independently with no backward pass. Because goodness is a single scalar per layer, multiclass inference requires running the network once per candidate label, and selecting the label that produces the highest accumulated goodness across layers: k forward passes for a k -class problem. Hinton additionally proposes learning a post-hoc softmax classifier over concatenated layer activities as a faster but explicitly suboptimal alternative, which itself requires standard backpropagation to train.

LPSL resolves this by replacing the scalar goodness with a full probability distribution over classes at each prediction head. Rather than a single value to be maximized or minimized, each head produces k competing logit scores whose relative magnitudes determine the predicted class — a richer signal that permits single-pass multiclass inference without auxiliary classifiers or multiple forward passes.

Extending FF to convolutional networks requires special treatment because convolutional layers lack a natural way to access label information at every spatial location. Scodellaro et al. [2023] introduced spatial label embedding strategies, including a Fourier-based scheme that encodes class labels as 2D spatial frequency patterns overlaid on the input. LPSL takes a simpler route: the prediction head flattens the output tensor of each segment (spatial reshaping for CNNs and pooling for transformers) and applies a standard linear classifier, requiring no architecture-specific label injection.

2.4 Predictive Coding Networks

Predictive coding [Rao & Ballard, 1999] originated as a theory of cortical processing in which higher areas predict the activity of lower areas and only residual prediction errors propagate forward. Whittington & Bogacz [2017] showed that this framework approximates backpropagation under certain conditions, and Millidge et al. [2022] reviews its development into a general-purpose local learning algorithm. Like LPSL, predictive coding achieves local credit assignment without a global backward pass. The key distinction is that predictive coding is grounded in a generative model

framework and relies on an iterative inference procedure at each layer, whereas LPSL retains standard gradient-based training within each segment and achieves locality purely through gradient blocking at segment boundaries.

2.5 Synthetic Gradients and Target Propagation

Jaderberg et al. [2017] proposed training auxiliary modules to predict the backpropagated gradient at each layer, allowing layers to update without waiting for the full backward pass. Unlike LPSL, this method still approximates the global gradient and requires training gradient-prediction networks that are themselves difficult to optimize. Target propagation methods [Bengio, 2014, Lee et al., 2015] compute layer-wise targets rather than gradients, providing an alternative local training signal. LPSL is simpler in that it requires neither target propagation nor gradient prediction; each segment trains as an isolated classifier on its own output representations.

3 Methodology

3.1 Network Segmentation

Let a neural network f consist of layers $\ell_1, \ell_2, \dots, \ell_L$ applied sequentially to an input $x \in \mathcal{X}$, producing logits over C classes. LPSL partitions these layers into K contiguous, non-overlapping segments $S_1, S_2, \dots, S_K$, where segment S_k consists of layers $\ell_{a_k}, \dots, \ell_{b_k}$, with $b_k + 1 = a_{k+1}$ for all $k < K$. The number of segments K and the placement of boundaries are treated as hyperparameters. The two extremes recover prior algorithms: $K = 1$ recovers backpropagation, and $K = L$ recovers per-layer local learning.

The forward pass of the network is unchanged. For a given input x , activations flow sequentially through all layers as in a standard network. Let h_k denote the output activations of the last layer in segment S_k .

3.2 Prediction Heads

Each segment S_k is assigned a **prediction head** P_k : a single fully-connected linear layer followed by softmax, mapping the flattened segment output $\hat{h}_k = \text{flatten}(h_k)$ to a probability distribution over C classes:

$$P_k(\hat{h}_k) = \text{softmax}(W_k \hat{h}_k + b_k), \quad W_k \in \mathbb{R}^{C \times d_k}, \quad b_k \in \mathbb{R}^C \quad (1)$$

where d_k is the dimensionality of $\hat{h}_k$. The prediction head is *external* to the network’s representational pathway: its output is never forwarded to subsequent segments. It exists solely to generate a local supervised signal for segment S_k .

Flattening conventions. For MLP segments, h_k is already a vector and no flattening is required. For CNN segments, h_k is a spatial feature map of shape (C_k, H_k, W_k) ; we flatten this to a vector of dimension $C_k \cdot H_k \cdot W_k$ for the prediction head while passing the unflattened tensor h_k to the next segment. For transformer segments, h_k is a sequence of token representations; we apply mean-pooling across the sequence dimension for encoder-style models, or take the representation of the final token for decoder-style models, before flattening. The network architecture itself is unmodified.

3.3 Local Loss and Gradient Isolation

Each segment S_k is trained with a cross-entropy loss between the prediction head output and the ground truth label y :

$$\mathcal{L}_k = \text{CrossEntropy}(P_k(\hat{h}_k), y) \quad (2)$$

Gradients are blocked at every segment boundary. Formally, let $\text{sg}(\cdot)$ denote the stop-gradient operator. The input to segment S_k for $k > 1$ is $\tilde{h}_{k-1} = \text{sg}(h_{k-1})$: a tensor whose value equals the output of the previous segment but which carries no gradient. Consequently, $\nabla_{\theta} \mathcal{L}_k$ is non-zero only for parameters θ belonging to segment S_k and its prediction head P_k . A single backward call from $\mathcal{L}_k$ updates all parameters of S_k and P_k jointly. In practice this is implemented via `.detach()` at each segment boundary in PyTorch.

This is functionally equivalent to treating each segment as an independent shallow network trained to classify from whatever input representation the previous segment provides. Critically, intra-segment gradient flow is unrestricted: layers within a segment can learn complex, jointly-optimized representations of arbitrary depth, constrained only at the boundary.

3.4 Independent Optimizers

Each segment S_k , together with its prediction head P_k , is assigned an independent optimizer instance. In all experiments we use Adam [Kingma & Ba, 2014] for both backpropagation and LPSL. Learning rates for backpropagation and LPSL are chosen independently via empirical search for best performance in each training regime; the network architecture and optimizer type are held fixed to ensure neither model has a computational advantage. Because gradients do not cross segment boundaries, optimizer states are fully decoupled: segment k ’s optimizer accumulates only gradients relevant to S_k . This decoupling opens the door to parallelizing segment updates across devices, though we do not exploit this in the current work.

3.5 Inference

At test time, the full network forward pass runs normally. The final prediction is taken from the last segment’s prediction head, $P_K(\hat{h}_K)$. Prediction heads of intermediate segments $P_1, \dots, P_{K-1}$ are discarded at inference.

Ensemble variants. We additionally experimented with combining predictions across all K heads at inference time. Specifically, we evaluated: (1) mode voting over per-head argmax predictions; (2) summing softmax probability vectors across heads and taking the argmax; and (3) weighted summation with either learned or manually assigned per-head scalars. None of these strategies improved over using only the final head P_K , and in some cases they degraded performance. We attribute this to intermediate heads being trained on partially-formed representations, making their predictions systematically less reliable than the final head. Ensemble inference also moves away from the local credit assignment motivation, since aggregating predictions across all depths of the network resembles the global goodness accumulation used in FF inference. We therefore adopt single final-head prediction as the default.

3.6 Relationship to Prior Algorithms

LPSL is a strict superset of both backpropagation and per-layer local learning:

- **Backpropagation** [Rumelhart et al., 1986] is recovered as the case $K = 1$. With a single segment spanning the full network, there are no segment boundaries, no stop-gradients, and the prediction head is simply the network’s output layer trained with cross-entropy — identical to standard backpropagation.
- **Per-layer local learning** [Nøkland & Eidnes, 2019] is recovered as the case $K = L$, with each individual layer forming its own segment and receiving isolated cross-entropy loss.
- **LPSL** generalizes both by allowing $1 \leq K \leq L$. Intermediate values of K trade off between the representational richness of longer intra-segment gradient paths and the locality of credit assignment. We observe qualitatively that deeper segments (more layers per segment, smaller K) yield better accuracy and more stable training across runs, consistent with the “short-sightedness” problem identified in single-layer local learning [Wang et al., 2021, Nøkland & Eidnes, 2019].

Non-differentiable components. Because gradients are blocked between segments, a non-differentiable component within one segment does not impede learning in any other segment. As long as each segment’s internal computation is differentiable, the algorithm proceeds normally regardless of what occurs at segment boundaries. This extends the class of trainable architectures relative to backpropagation.

4 Experiments

We evaluate LPSL on three tasks spanning distinct architectures: a synthetic cluster classification task with a multilayer perceptron (MLP), image classification on MNIST with a convolutional neural network (CNN), and a modular addition task with a small transformer. In each experiment, we compare LPSL against a backpropagation baseline using the same architecture and optimizer type. Performance is reported as test set accuracy. Results are presented as mean and standard deviation across multiple independent training runs from different random initializations.

4.1 Synthetic Cluster Classification (MLP)

Task. We construct a synthetic classification dataset with 5 classes, each defined by 5 Gaussian clusters in a 64-dimensional feature space. The resulting class structure is multimodal and non-linearly separable, providing a controlled but non-trivial classification problem.

Architecture and training. Both models (Figure 1) are trained with the Adam optimizer using independently tuned learning rates. Performance against the test set is averaged across 10 runs, where both models receive the same batches during training for a given run.

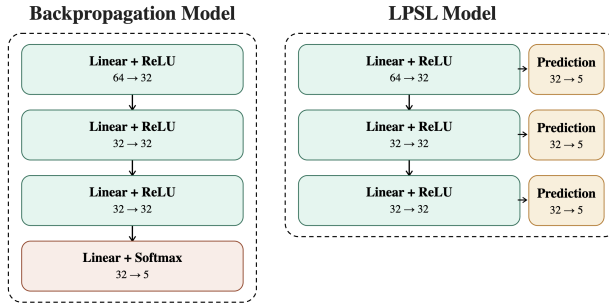


Figure 1: MLP Architecture for Backpropagation and LPSL Models.

Results. We find both LPSL and backpropagation converge to approximately 80-85% test accuracy (Figure 2). The training dynamics differ notably. Backpropagation reaches peak performance quickly — around 25 training epochs — but then exhibits a visible decline, dropping to roughly 81%, consistent with overfitting to the training distribution. LPSL converges more slowly, reaching approximately 80% at the point where backpropagation peaks, but continues to improve and plateaus near 85% after approximately 150 epochs with no visible overfitting. This suggests that the local loss structure may act as an implicit regularizer. Per-epoch wall-clock time is comparable between both training regimes on this task.

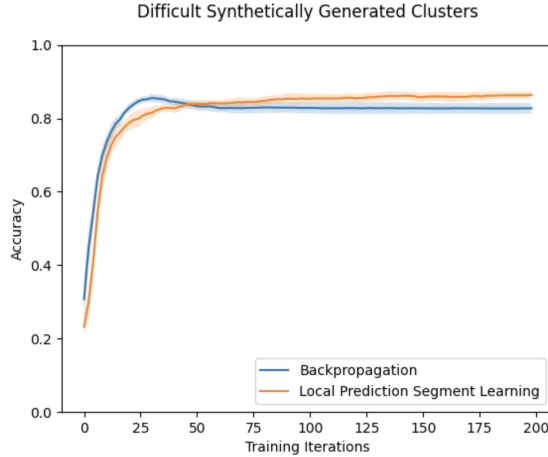


Figure 2: Mean and standard deviation of test accuracy for BP and LPSL on synthetic clusters.

4.2 MNIST Image Classification (CNN)

Task. We train on MNIST [LeCun et al., 1998], a 10-class handwritten digit classification benchmark. While MNIST is a well-studied benchmark, it provides a controlled setting for evaluating whether LPSL can effectively train a convolutional architecture under local credit assignment.

Architecture and training. Both models (Figure 3) are trained with the Adam optimizer using independently tuned learning rates. Performance against the test set is averaged across 10 runs, where both models receive the same batches during training for a given run.

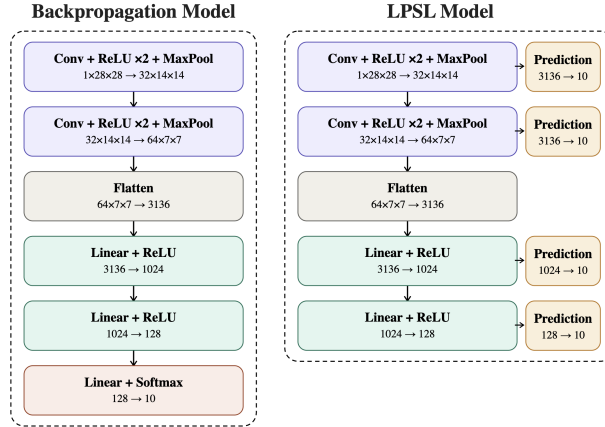


Figure 3: CNN Architecture for Backpropagation and LPSL Models.

Results. Backpropagation trains reliably and converges to approximately 98% test accuracy within roughly 100 training epochs (Figure 4). LPSL is substantially more variable across runs: mean test accuracy rises gradually from approximately 60% at epoch 50 to approximately 95% by epoch 200, with high standard deviation throughout, indicating that individual runs vary considerably in convergence speed. By epoch 400, most LPSL runs have converged to approximately 98% test accuracy and standard deviation is minimal, nearly matching backpropagation’s final performance at the cost of approximately $4\times$ more training epochs and greater run-to-run variance.

We attribute the high variance primarily to sensitivity to initialization and the compounding effects of local credit assignment: if an early segment fails to learn useful representations, later segments receive poor inputs and may struggle to converge. This “short-sightedness” problem is a known challenge in

local learning [Wang et al., 2021], and deeper segments may partially mitigate it by providing richer intra-segment gradient signal. We observe qualitatively that increasing segment depth improves both accuracy and stability, though we defer a systematic ablation of segment placement to future work.

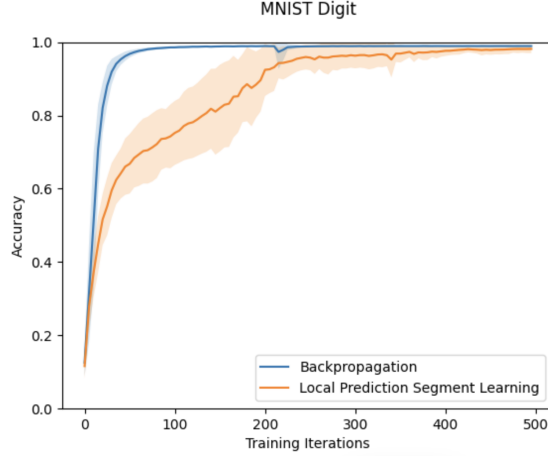


Figure 4: Mean and standard deviation of test accuracy for BP and LPSL on MNIST.

4.3 Modular Addition (Transformer)

Task. Modular addition is a synthetic arithmetic task in which a model must predict $(a + b) \bmod p$ given integers $a, b \in \{0, \dots, p - 1\}$ for a fixed modulus p . We frame this as a p -class classification problem, using it as a controlled probe of whether LPSL can successfully train a transformer to learn a non-trivial algorithmic relationship and generalize to held-out pairs.

Architecture and training. Both models (Figure 5) are trained with the Adam optimizer using independently tuned learning rates. Performance against the test set is averaged across 10 runs, where both models receive the same batches during training for a given run.

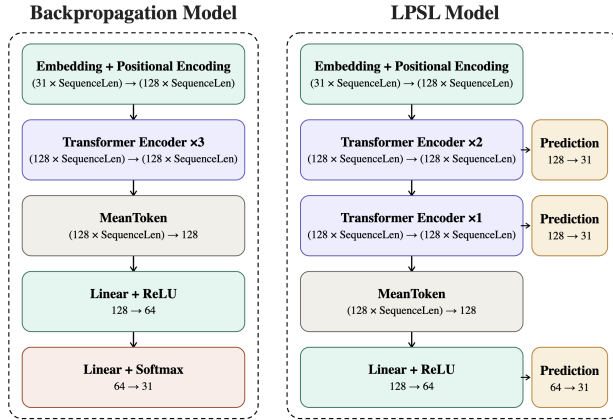


Figure 5: Transformer Architecture for Backpropagation and LPSL Models.

Results. Both backpropagation and LPSL achieve approximately 95% test accuracy (Figure 6). LPSL converges substantially faster on average; however, backpropagation ultimately achieves marginally better accuracy in the long run. LPSL again exhibits significant run-to-run variance, with some runs converging slowly, whereas backpropagation converges more uniformly.

The faster average convergence of LPSL on this task, in contrast to its slower convergence on the CNN task, may reflect properties of the transformer architecture that interact favorably with local

credit assignment; specifically the relatively shallow depth per segment and the dense representational mixing within attention layers. We treat this as a preliminary observation and note that a more thorough investigation across transformer scales and moduli is needed.

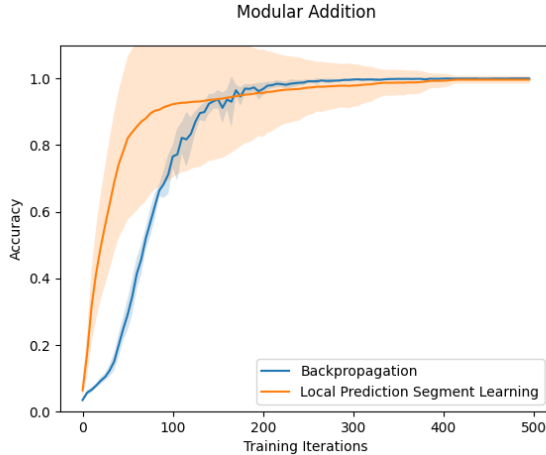


Figure 6: Mean and standard deviation of test accuracy for BP and LPSL on modular addition.

5 Conclusion

We introduced Local Prediction Segment Learning (LPSL), a training algorithm that partitions a neural network into contiguous segments and trains each with an isolated cross-entropy loss produced by a lightweight auxiliary prediction head. LPSL achieves approximate local credit assignment, supports multiclass output natively in a single forward pass, and applies without modification to MLPs, convolutional networks, and transformers. It is a strict superset of both backpropagation and per-layer local learning, recovering each as a limiting case.

On three controlled tasks spanning three distinct architectures, LPSL successfully trained models achieving accuracy comparable to backpropagation. On synthetic cluster classification, LPSL matched final accuracy while exhibiting less overfitting. On MNIST, LPSL ultimately matched backpropagation’s accuracy at the cost of greater convergence variance. On modular addition with a transformer, LPSL roughly matched backpropagation’s final accuracy while converging faster on average — a result that, while preliminary, suggests the approach is not uniformly slower than backpropagation and may have architecture-dependent convergence advantages.

We are candid about the limitations of the current work. Although LPSL succeeds on these controlled examples and in some cases outperforms backpropagation in specific training dynamics, all experiments are on small-scale, controlled tasks, and we do not claim that LPSL safely generalizes to arbitrary architectures or real-world problem scales. Run-to-run variance under LPSL is consistently higher than under backpropagation, and sensitivity to macro-hyperparameters such as learning rate, segment count, and segment placement is greater. These are practical limitations that would need to be addressed before LPSL could serve as a drop-in replacement for backpropagation.

Nonetheless, we view these results as a meaningful proof of concept in the space of local credit assignment. The ability to train non-trivial models across MLPs, CNNs, and transformers under a locally constrained learning regime, using the same standard cross-entropy objective and gradient machinery as backpropagation, and with no architecture-specific adaptations, represents a useful step toward more general and biologically motivated training algorithms. Future work should address systematic study of segment placement strategies, scaling to larger tasks and architectures, theoretical analysis of convergence under local credit assignment, and the potential for parallel segment training as a practical efficiency benefit.

References

- Bengio, Y., Lamblin, P., Popovici, D., & Larochelle, H. (2007). Greedy layer-wise training of deep networks. *Advances in Neural Information Processing Systems*, 19.
- Bengio, Y. (2014). How auto-encoders could provide credit assignment in deep networks via target propagation. *arXiv preprint arXiv:1407.7906*.
- Bengio, Y., Lee, D.-H., Bornschein, J., Mesnard, T., & Lin, Z. (2015). Towards biologically plausible deep learning. *arXiv preprint arXiv:1502.04156*.
- Belilovsky, E., Eickenberg, M., & Oyallon, E. (2019). Greedy layerwise learning can scale to ImageNet. *International Conference on Machine Learning*, pp. 583–593.
- Belilovsky, E., Eickenberg, M., & Oyallon, E. (2020). Decoupled greedy learning of CNNs. *International Conference on Machine Learning*, pp. 736–745.
- Crick, F. (1989). The recent excitement about neural networks. *Nature*, 337(6203), 129–132.
- Hinton, G. (2022). The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*.
- Jaderberg, M., Czarnecki, W. M., Osindero, S., Vinyals, O., Graves, A., Silver, D., & Kavukcuoglu, K. (2017). Decoupled neural interfaces using synthetic gradients. *International Conference on Machine Learning*, pp. 1627–1635.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Lee, D.-H., Zhang, S., Fischer, A., & Bengio, Y. (2015). Difference target propagation. *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 498–515.
- Lillicrap, T. P., Cownden, D., Tweed, D. B., & Akerman, C. J. (2016). Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7, 13276.
- Millidge, B., Salvatori, T., Song, Y., Bogacz, R., & Lukasiewicz, T. (2022). Predictive coding: Towards a future of deep learning beyond backpropagation? *International Joint Conference on Artificial Intelligence*, pp. 5538–5545.
- Nøkland, A., & Eidnes, L. H. (2019). Training neural networks with local error signals. *International Conference on Machine Learning*, pp. 4839–4850.
- Ortiz Torres, M., Lange, M., & Raulf, A. P. (2025). On advancements of the Forward-Forward algorithm. *arXiv preprint arXiv:2504.21662*.
- Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: A functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Scodellaro, R., Kulkarni, A., Alves, F., & Schröter, M. (2023). Training convolutional neural networks with the Forward-Forward algorithm. *arXiv preprint arXiv:2312.14924*.
- Wang, M., Guo, H., Jiang, T., Li, Z., Yu, Y., Shu, T., & Shi, Z. (2021). Revisiting locally supervised learning: An alternative to end-to-end training. *International Conference on Learning Representations*.
- Whittington, J. C. R., & Bogacz, R. (2017). An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation*, 29(5), 1229–1262.